

네트워크보안

TLS 1.2 AES-GCM Nonce 재사용을 이용한 Record 위조

2026년 7월

결과 요약

제공된 `tls_live.pcap`의 서버 방향 TLS 1.2 통신은 TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256을 사용한다. 서버 application data 1-3번이 같은 8바이트 explicit nonce를 재사용하므로, 같은 write key와 4바이트 fixed salt 아래에서 전체 96비트 GCM nonce가 충돌한다. 이 충돌로 CTR keystream을 재사용해 `amt=0100`을 `amt=0500`으로 바꾸고, tag 차분에서 GHASH subkey를 복원해 유효한 새 tag를 계산했다.

최종 제출 record는 77바이트이다. 아래 두 줄을 개행 없이 연결한 값을 제출한다.

```
17030300489c269a9f29810ab0a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc3
6a29772edbf159643b468f1f1e520e7341daee867a49665542cd9f01f5a9c906b73ba3e13bbfea0e98a18
```

1 PCAP 분석과 취약점 식별

1.1 TCP/TLS record 복원

pcap의 link type은 DLT_IPV4(228)이다. 서버 방향 flow는 다음과 같다.

10.0.0.1:443 -> 10.0.0.2:54321

TCP segment를 sequence number 순으로 정렬하고 overlap을 제거해 byte stream을 재조립했다. 5바이트 TLS header의 길이 필드로 record를 분리하고 ServerHello에서 cipher suite 0xc02f를 확인했다.

ChangeCipherSpec 뒤 새 server write state의 TLS sequence number는 0으로 초기화된다. encrypted Finished가 `seq=0`이고, 이어지는 application data의 sequence number는 1-4이다.

App record	TLS seq	Ciphertext	Fragment	Explicit nonce
1	1	34	58	9c269a9f29810ab0
2	2	50	74	9c269a9f29810ab0
3 (target)	3	48	72	9c269a9f29810ab0
4	4	40	64	9c269a9f29810ab1

1.2 TLS 1.2 GCM nonce와 AAD

TLS 1.2 AES-GCM nonce는 write state의 4바이트 fixed salt와 record의 8바이트 explicit nonce를 연결한 값이다.

$$nonce = write_salt[4] \parallel nonce_explicit[8].$$

세 record는 같은 연결, 방향, cipher epoch에 있으므로 key와 salt가 같다. 따라서 같은 explicit nonce는 같은 전체 nonce, 같은 CTR keystream, 같은 $S = E_K(J_0)$ 를 뜻한다.

TLS AEAD additional data는 다음 13바이트다.

$$A = seq[8] \parallel type[1] \parallel version[2] \parallel plaintext_length[2].$$

목표 record의 AAD는 다음과 같다.

```
000000000000000003 || 17 || 0303 || 0030
= 0000000000000000031703030030
```

AAD의 0030은 48바이트 평문 길이이고, record header의 0048은 explicit nonce(8)+ciphertext(48)+tag(16)=72바이트 wire fragment 길이이므로 서로 다르다. (GCM은 CBC와 달리 별도 padding이나 HMAC 없이 GHASH 내부에서만 zero padding을 쓴다.)

2 GHASH Authentication Subkey 복원

2.1 Nonce 재사용 방정식

GCM에서

$$H = E_K(0^{128}), \quad T_i = S \oplus \text{GHASH}_H(A_i, C_i)$$

이다. 같은 nonce인 record 두 개의 tag를 XOR하면 S 가 제거된다.

$$T_1 \oplus T_2 = \text{GHASH}_H(A_1, C_1) \oplus \text{GHASH}_H(A_2, C_2).$$

GHASH는

$$\mathbb{F}_{2^{128}} = \mathbb{F}_2[x]/(x^{128} + x^7 + x^2 + x + 1)$$

에서 계산한다. block을 $X_1, \dots, X_m$ 이라 하면

$$Y_0 = 0, \quad Y_i = (Y_{i-1} \oplus X_i)H$$

이므로 각 GHASH는 H 에 대한 다항식 $P_i(H)$ 이다. 2번 ciphertext는 50바이트라서 AAD 1 block, ciphertext 4 blocks, length 1 block을 사용한다. 따라서 1번과 2번 record로 만든

$$F_{12}(H) = P_1(H) \oplus P_2(H) \oplus T_1 \oplus T_2 = 0$$

은 6차 방정식이다.

2.2 유한체 근 분리와 검증

solver는

$$\gcd(F_{12}(X), X^{2^{128}} - X)$$

로 $\mathbb{F}_{2^{128}}$ 안의 선형 인수만 분리한 뒤 근을 나눈다. NIST GCM은 field element를 MSB-first로 표현하지만 다항식 연산은 bit 0을 상수항 계수로 쓰므로, 표현 경계에서 128비트 bit reverse를 적용해 다음 대응을 사용한다.

```
NIST 0x80000000000000000000000000000000 <-> polynomial integer 1
NIST 0x40000000000000000000000000000000 <-> polynomial integer 2 (= x)
```

solver는 첫 번째 값을 GCM 곱셈 항등원으로 사용해 임의 probe와 곱한 결과가 probe 그대로인지 실행 시 self-test한다. 1번과 2번 record로 만든 6차식에서 체 내부 root는 1개였고, 3번 record의 동일 S 조건까지 통과한 후보도 1개였다.

검증 단계	후보 수
F_{12} 의 $\mathbb{F}_{2^{128}}$ root	1
3번 record의 동일 S 조건 통과	1

각 후보는 재사용 record 세 개 모두에서

$$T_i \oplus P_i(H) = S$$

가 같은지 검사했다. 유일하게 남은 값은 다음과 같다.

```
H          = 112332a84132bc0c5c23a61037723683
E_K(J0)    = 1e023d8461003896981f1441c5feab90
```

재사용 nonce에서 GCM authentication subkey H 와 S 를 모두 얻었으므로, 그 nonce에 대한 새 ciphertext와 tag를 만들 수 있다.

3 평문 및 Tag 위조

3.1 Ciphertext 변경

원래 평문은 문제 본문에 주어진 known plaintext다. 두 평문은 모두 48바이트이고 0-based offset 32의 한 바이트만 다르다.

```
old: action=set_salary&uid=0007&amt=0100&month=202603
new: action=set_salary&uid=0007&amt=0500&month=202603
```

'1'= $0x31$, '5'= $0x35$ 이므로 delta는 $0x04$ 다. CTR에서 $C = P \oplus KS$ 이므로

$$C' = C \oplus P \oplus P'$$

로 바꾸면 된다. ciphertext의 해당 byte는 $f5 \text{ xor } 04 = f1$ 이 된다.

```
old C = a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc36a29772edbf159643b468f5f1e520e7341daee867a49665542cd9
new C = a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc36a29772edbf159643b468f1f1e520e7341daee867a49665542cd9
```

3.2 Tag 재계산

nonce, AAD, 길이가 그대로이므로 S 도 같다.

$$S = T_{\text{old}} \oplus \text{GHASH}_H(A, C_{\text{old}}),$$

$$T_{\text{new}} = S \oplus \text{GHASH}_H(A, C_{\text{new}}).$$

```
old tag = b0d6e0ac2be205621f7ea4a034519f95
GHASH old = aed4dd284ae23df48761b0e1f1af3405
S          = 1e023d8461003896981f1441c5feab90
GHASH new = ee1d6718f16b4b2ca60cafbf65172188
new tag    = f01f5a9c906b73ba3e13bbfea0e98a18
```

4 최종 Record 구조

Offset	Size	Field	Value
0	1	ContentType	17
1-2	2	TLS version	0303
3-4	2	Fragment length	0048
5-12	8	Explicit nonce	9c269a9f29810ab0
13-60	48	Forged ciphertext	a99141...5542cd9
61-76	16	GCM tag	f01f5a9c906b73ba3e13bbfea0e98a18

fragment는 $8 + 48 + 16 = 72$ 바이트이고 header를 포함한 record는 $5 + 72 = 77$ 바이트, 즉 개행을 제외한 hex 154문자다.

5 구현과 재현 검증

구현 파일은 `solve_tls_gcm_nonce_reuse.py`다. Python 표준 라이브러리만 사용하며 pcap magic/endian/link type과 packet 길이를 검사하고, TCP segment를 sequence number로 재조립하며, TLS record 경계를 검증한다. $GF(2^{128})$ 곱셈, 역원, 다항식 GCD와 인수분해도 직접 구현했다. 인수분해 난수 seed는 3으로 고정하고 최종 후보는 세 tag 방정식으로 결정론적으로 검증한다.

이 제출 폴더의 src 디렉터리 안에서 다음을 실행한다.

```
python3 solve_tls_gcm_nonce_reuse.py
```

코드는 다음 조건을 모두 assertion으로 검사한다.

- cipher suite, TLS sequence number, nonce 재사용 pattern;
- 유일한 H 와 세 record에서 동일한 S ;
- 새 ciphertext가 재사용 keystream 아래 목표 평문으로 복원됨;
- $T_{\text{new}} \oplus \text{GHASH}_H(A, C_{\text{new}}) = S$;
- fragment/record 길이와 최종 예상 hex; 그리고
- 생성값과 `forged_record.txt`의 일치.

6 참고 자료

- IETF RFC 5246, *The Transport Layer Security (TLS) Protocol Version 1.2*, Section 6.2.3.3.
- IETF RFC 5288, *AES Galois Counter Mode (GCM) Cipher Suites for TLS*, Section 3.
- NIST SP 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*.
- David A. McGrew and John Viega, *The Security and Performance of the Galois/Counter Mode (GCM) of Operation*, 2004.
- Antoine Joux, *Authentication Failures in NIST version of GCM*, 2006.